Lab Report 4: ROS 2 Launch, Rosbag, and rqt-plot Visualization

Submitted by: Syeda Imaan Fatima 2022-MC-28

Course: Mobile Robotics

Environment: Ubuntu Linux (ROS 2 Humble)

Workspace: ~/ros2_ws_imaan

1. Objective

The primary objective of this lab session was to master the ROS 2 orchestration and diagnostic ecosystem. This involved:

- Automating multi-node startup using **Launch files**.
- Capturing and replaying real-time sensor/actuator data using **Rosbag**.
- Analyzing control signals through graphical visualization with **rqt_plot**.
- Implementing an autonomous **Follow-the-Leader** control law between two mobile agents.

2. Prerequisites & Environment Setup

Before execution, the Linux environment was prepared to ensure all ROS 2 Humble dependencies were met.

1. **Sourcing the Environment:** The ROS 2 setup script was sourced to provide access to CLI tools.

```
source /opt/ros/humble/setup.bash
```

2. **Package Installation:** Necessary packages for simulation and data recording were verified/installed:

```
sudo apt install ros-humble-turtlesim ros-humble-rosbag2 ros-humble-rqt-plot
```

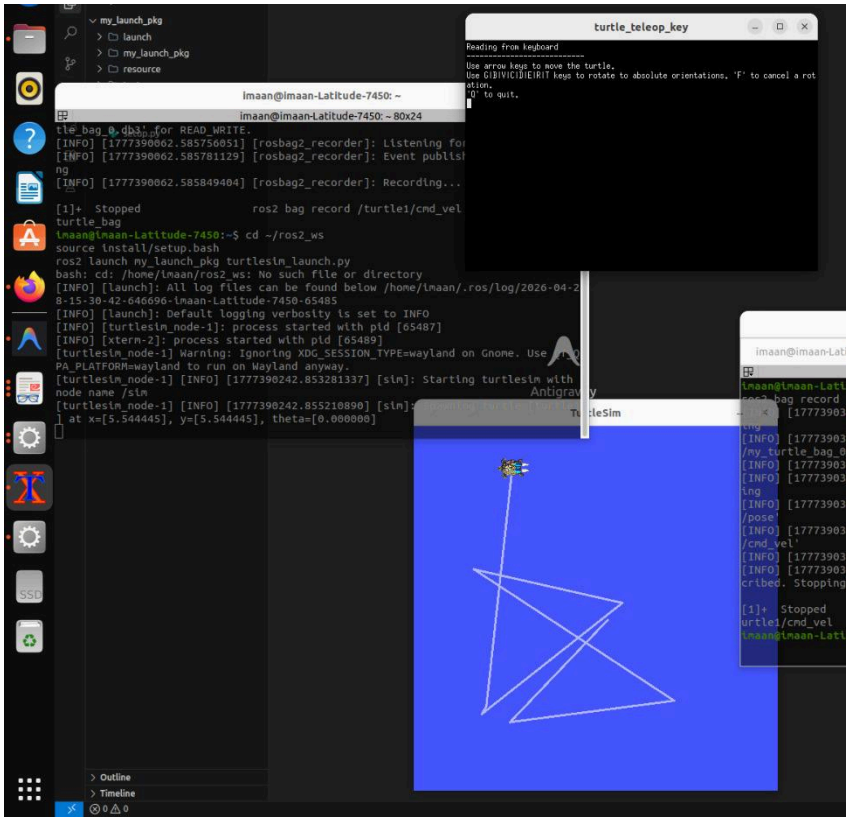
3. **Workspace Construction:** A dedicated package my_launch_pkg was created with ament_python build type to house the launch and node scripts.

3. Step-by-Step Implementation

Step A: Launching the Simulation

I developed a launch file, turtlesim_launch.py, to initialize the simulation environment. This replaced the need for multiple terminal windows by starting both the simulator and the teleop key node simultaneously.

- **Build & Source:** ```bash
  colcon build --packages-select my_launch_pkg
  source install/setup.bash
  ros2 launch my_launch_pkg turtlesim_launch.py



Step B: Rosbag Recording & Data Extraction

To analyze the turtle's trajectory, I recorded the /turtle1/pose and /turtle1/cmd_vel topics.

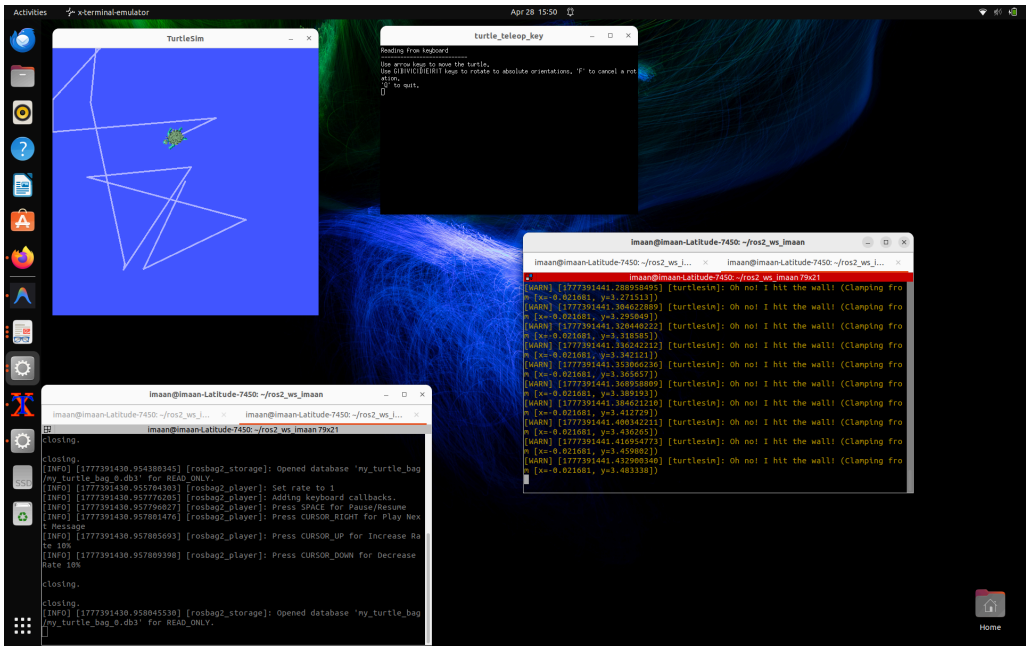
- **Recording:** ros2 bag record /turtle1/pose /turtle1/cmd_vel -o my_turtle_bag
- **Playback:** After killing the nodes, I replayed the movement using ros2 bag play my_turtle_bag.
- **Extraction:** The pose data was echoed into a text file for reporting:

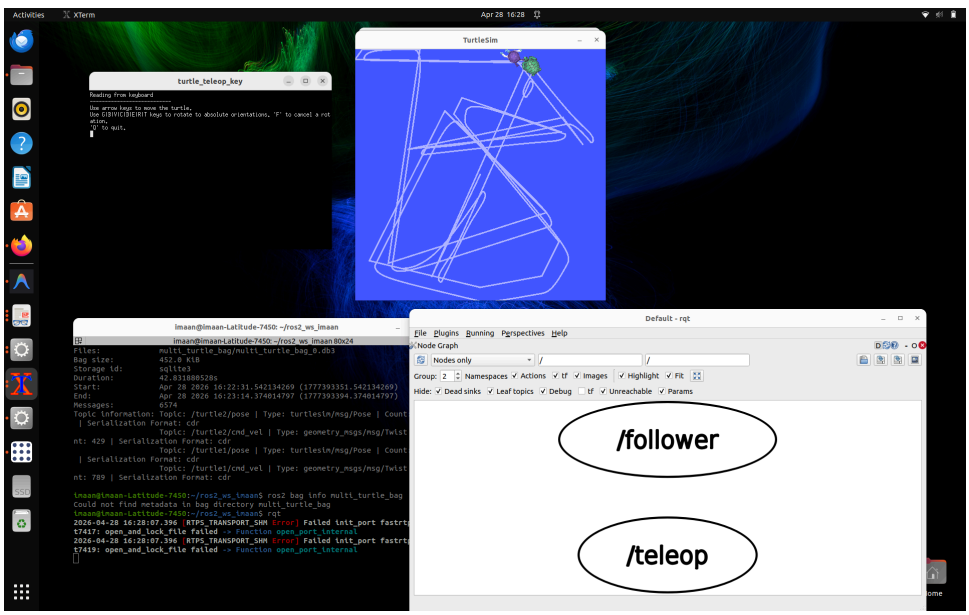
  ros2 topic echo /turtle1/pose --bag my_turtle_bag > turtle1_trajectory.txt

Step C: Follow-the-Leader Behavior

I implemented the follower_node.py to create a leader-follower dynamic between two turtles.

1. **Logic:** The node calculates the Euclidean distance (error) between turtle1 and turtle2.
2. **Control Law:** A **Proportional (P) Controller** was used to adjust the follower's velocity based on its distance from the leader.
3. **Execution:** The system was launched via follow_the_leader_launch.py, which handled spawning turtle2 at specific coordinates $(2.0, 2.0)$ and running the tracking logic.

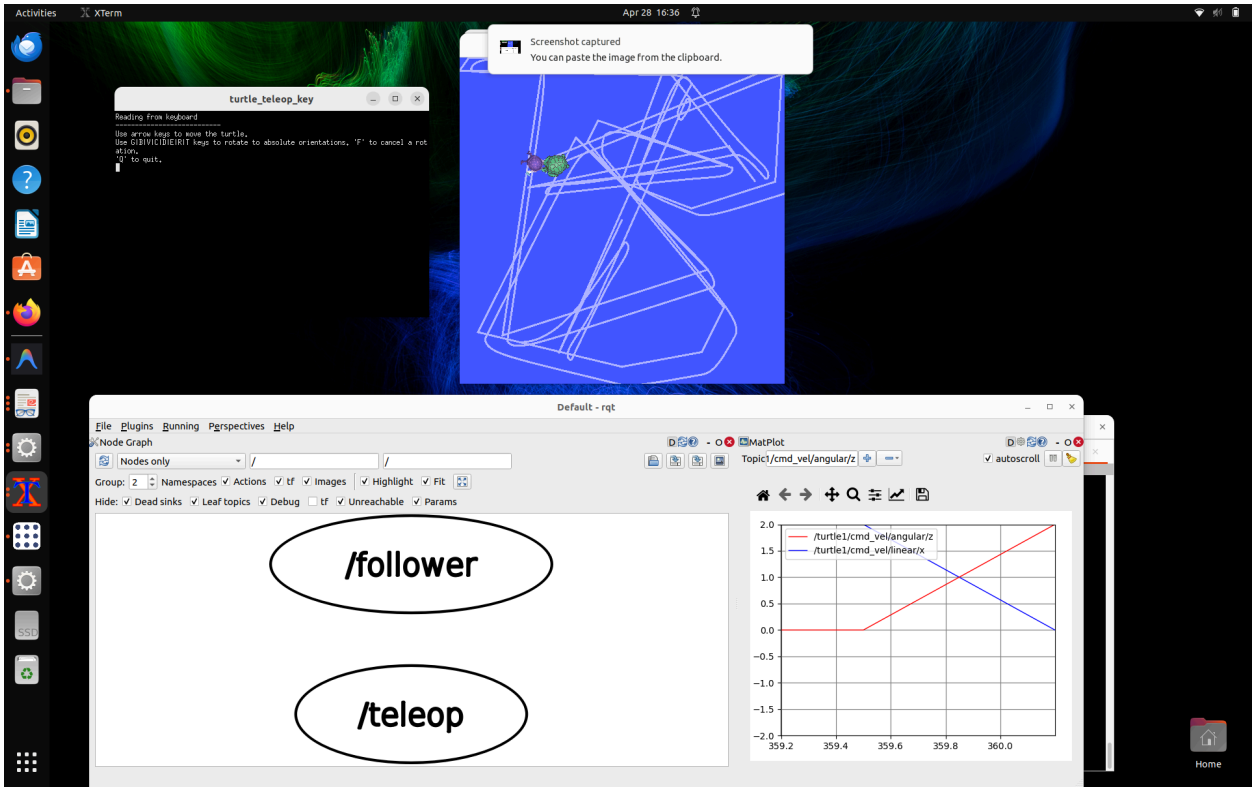




Step D: Real-Time Visualization (rqt_plot)

Using the rqt GUI, I monitored the velocity commands sent to the turtles.

- **Topic Tracked:** /turtle1/cmd_vel/linear/x and /turtle1/cmd_vel/angular/z.
- **Observation:** The plot provided a clear visual of the control inputs, allowing me to see the "stepped" nature of the keyboard teleoperation commands.



4. Key Technical Findings

- **Subscriber vs. Publisher:** During the Rosbag replay, I observed that replaying the pose topic does not move the turtle because the turtlesim_node only *publishes* pose; it does not *subscribe* to it. Movement replay requires the /cmd_vel topic.
- **Launch Utility:** Utilizing data_files in setup.py is critical for ensuring colcon correctly installs the launch directory into the share folder.
- **P-Controller Tuning:** High angular gains ($K_a = 6.0$) allowed for sharp turning, while the linear gain ($K_p = 1.5$) ensured a smooth approach to the leader.

5. Conclusion

The lab successfully integrated the core diagnostic and automation tools of ROS 2. For a mechatronics application, the ability to record raw data (Rosbag) and visualize control responses (rqt) is vital for debugging hardware-in-the-loop systems.